

Introduction

This tutorial will guide you through the process of creating a first Zynq design using the Vivado™ Integrated Development Environment (IDE), and introduce the IP Integrator environment for the generation of a simple Zynq processor design to be implemented on the ZedBoard. The Software Development Kit (SDK) will then be used to create a simple software application which will run on the Zynq's ARM Processing System (PS) to control the hardware that is implemented in the Programmable Logic (PL).

The tutorial is split into three exercises, and is organised as follows:

Exercise 1A - This exercise will guide you through the process of launching Vivado IDE and creating a project for the first time. The various stages of the *New Project Wizard* will be introduced.

Exercise 1B - In this exercise, we will use the project that was created in Exercise 1A to build a simple Zynq embedded system with the graphical tool, IP Integrator, and incorporating existing IP from the Vivado IP Catalog. A number of design aids will be used throughout this exercise, such as the Board Automation feature which automates the customisation of IP modules for a specified device or board; in this case we will be using the ZedBoard Zynq Evaluation and Development Kit. The Designer Assistance feature, which assists with the connections between the Zynq PS and the IP modules in the PL will also be demonstrated.

Once the design is finished, a number of stages will be undertaken to complete the hardware system and generate a bitstream for implementation in the PL. The completed hardware design will then be exported to the Software Development Kit (SDK) for the development of a simple software application in **Exercise 1C**.

Exercise 1C - In this short third exercise, the SDK will be introduced, and a short software application will be created to allow the Zynq processor to interact with the IP implemented in the PL. A connection to the hardware server that allows the SDK to communicate with the Zynq processors will be established. The software drivers that are automatically created by the Vivado IDE for IP modules will be explored and integrated into the software application, before finally building and executing the software application on the ZedBoard.

NOTE: Throughout all of the practical tutorial exercise we will be using C:\Zynq_Book as the working directory. If this is not suitable, you can substitute it for a directory of your choice, but you should be aware that you will be required to make alterations to some source files in order for exercises to complete successfully.

Exercise 1A Creating a First IP Integrator Design

In this exercise we will create a new project in Vivado IDE by moving through the stages of the Vivado IDE *New Project Wizard*.

We will start by launching the Vivado IDE.

- (a) Launch Vivado by double-clicking on the Vivado desktop icon: , or by navigating to **Start > All Programs > Xilinx Design Tools > Vivado 2014.1 > Vivado 2014.1**
- (b) When Vivado loads, you will be presented with the *Getting Started* screen as in Figure 1.1.



Figure 1.1: Vivado IDE Getting Started Screen

- (c) Select the option to **Create New Project** and the *New Project Wizard* will open, as in Figure 1.2.

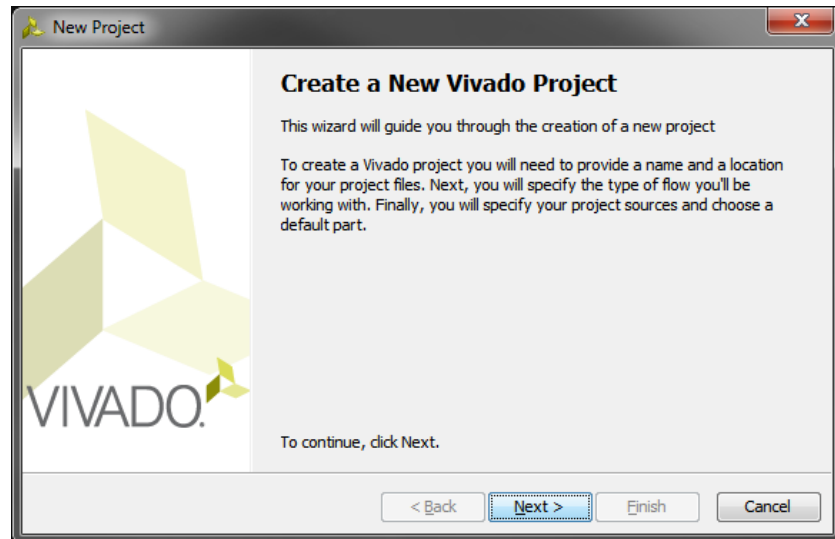
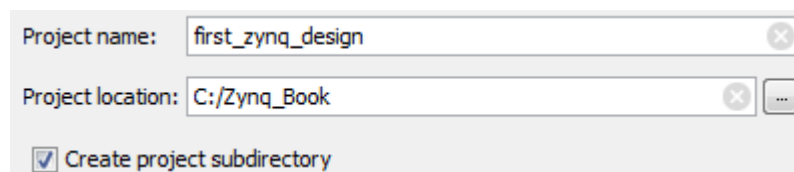


Figure 1.2: New Project Dialogue

Click **Next**.

- (d) At the Project Name dialogue, enter **first_zynq_design** as the **Project name** and **C:/Zynq_Book** as **Project location**.

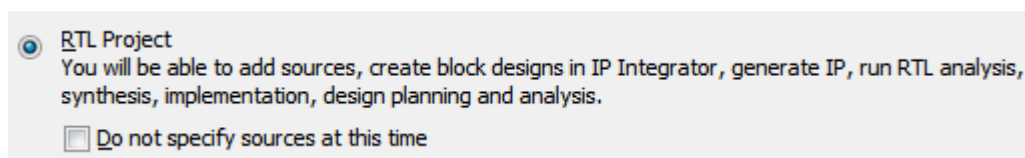
Make sure that you select the option to **Create project subdirectory**. All options should be the same as shown below:



Click **Next**.

A directory named **Zynq_Book** will be created on your **C drive** if it did not already exist.

- (e) At the *Project Type* dialogue, select **RTL Project** and ensure that the option **Do not specify sources at this time** is not selected:



Click **Next**.

- (f) Select **VHDL** as the **Target language** in the *Add Sources* dialogue.

If existing sources, in the form of HDL or netlist files, were to be added to the project they could be imported at this stage.

As we do not have any sources to add to the project, click **Next**.

(g) The *Add Existing IP (optional)* dialogue will open.

If existing IP sources were to be included in the project, they could be added here.

As we do not have any existing IP to add, click **Next**.

(h) The *Add Constraints (optional)* dialogue will open.

This is the stage where any physical or timing constraints files could be added to the project.

As we do not have any constraints files to add, click **Next**.

(i) From the *Default Part* dialogue, select **Boards** from the *Specify* box and select **ZedBoard Zynq Evaluation and Development Kit** from the *Display Name* list and **All** from the *Board Rev* list, as shown in Figure 1.3. Select the appropriate revision for your board (in this case **Rev. C** has been selected).

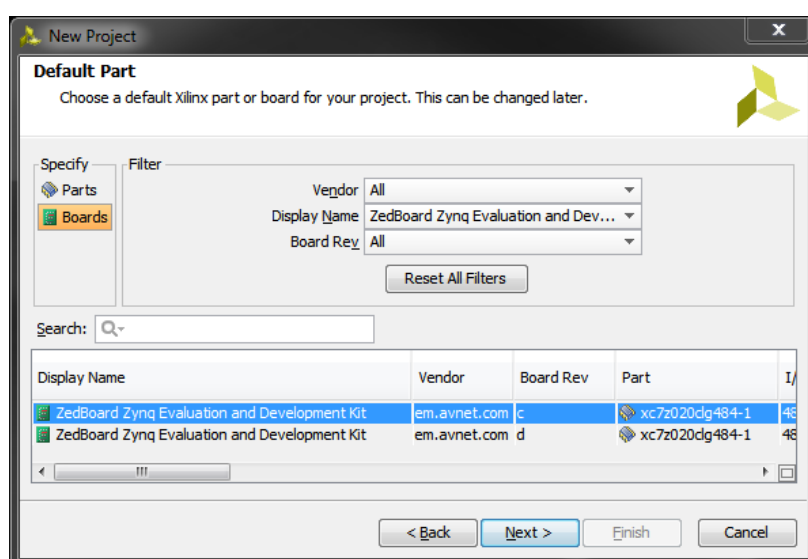


Figure 1.3: Default Part Dialogue Options

Click **Next**.

(j) In the *New Project Summary* dialogue, review the specified options, and click **Finish** to create the project.

Now that we have created our first project in Vivado IDE, we can now move on to creating our first Zynq embedded system design.

Before doing that, the Vivado IDE tool layout should be introduced. The default Vivado IDE environment layout is shown in Figure 1.4 (other layouts can be chosen by selecting different perspectives).

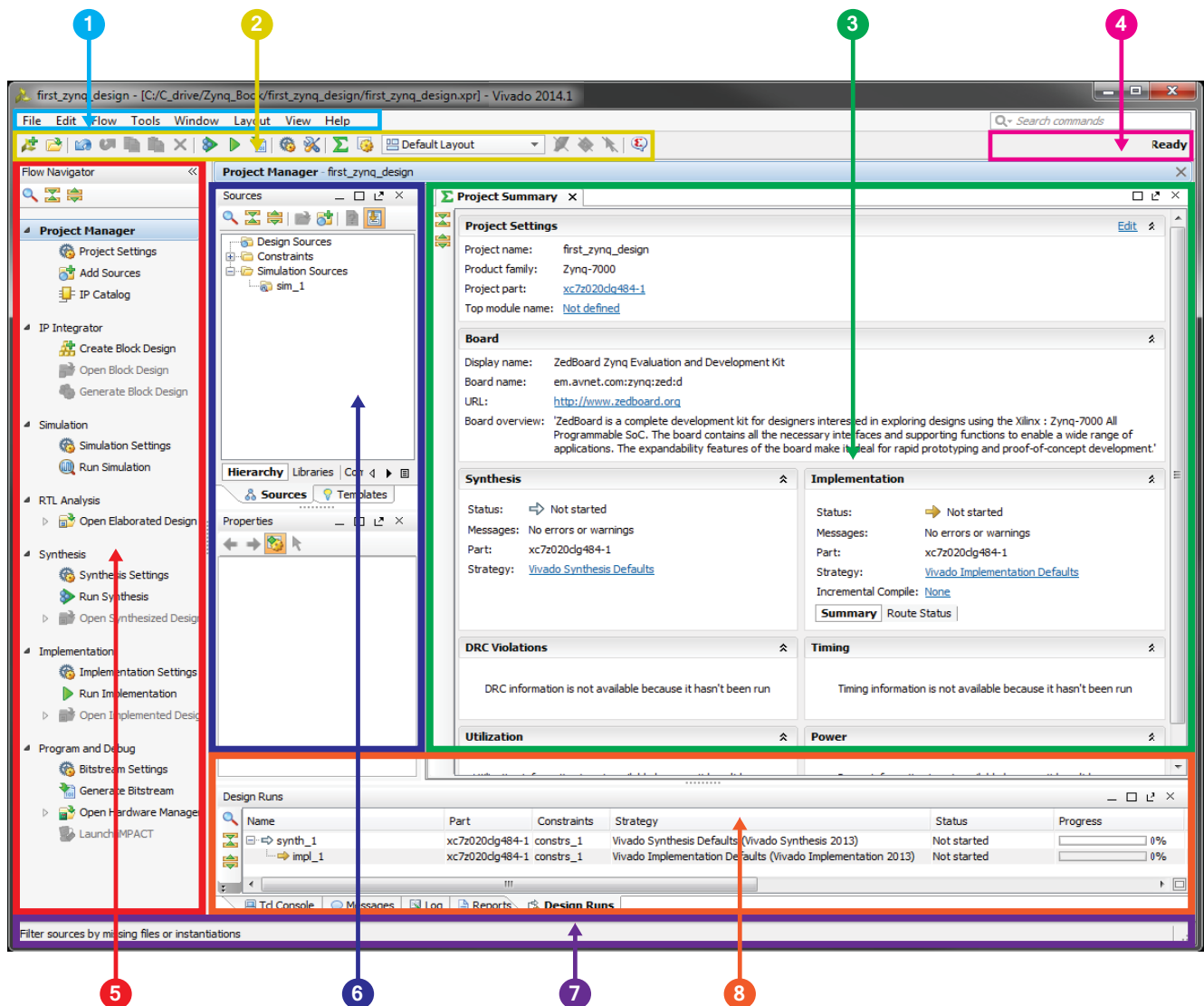


Figure 1.4: Vivado IDE Environment Layout

With reference to the numbered labels in Figure 1.4, the main components of the Vivado IDE environment are:

1. **Menu Bar** - The main access bar gives access to the Vivado IDE commands.
2. **Main Toolbar** - The main toolbar provides easy access to the most commonly used Vivado IDE commands. Tooltips that provide information for each command on the toolbar can be accessed

by hovering the mouse pointer over the corresponding button, as shown in Figure 1.5.

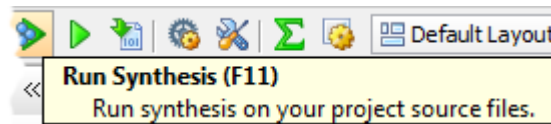


Figure 1.5: Toolbar tooltips

3. **Workspace** - The workspace provides a larger area for panels which require a greater screen space and those with a graphical interface, such as:
 - Schematic panel
 - Device panel
 - Package panel
 - Text editor panel
4. **Project Status Bar** - The project status bar displays the status of the currently active design.
5. **Flow Navigator** - The Flow Navigator provides easy access to the tools and commands that are necessary to guide your design from start to finish, starting in the *Project Manager* section with design entry and ending with bitstream generation in the *Program and Debug* section. Run commands are available in the *Simulation*, *Synthesis* and *Implementation* sections to simulate, synthesise and implement the active design.
6. **Data Windows Pane** - The Data Windows pane, by default, displays information that relates to design data and sources, including:
 - **Properties window** - Shows information about selected logic objects or device resources.
 - **Netlist window** - Provides a hierarchical view of the synthesised or elaborated logic design.
 - **Sources window** - Shows IP Sources, Hierarchy, Libraries and Compile Order views.
7. **Status Bar** - The status bar displays a variety of information, including:
 - Detailed information regarding menu bar and toolbar commands will be shown in the lower left side of the status bar when the command is accessed.
 - When hovering over an object in the Schematic window with the mouse pointer, the object details appear in the status bar.
 - During constraint and placement creation in the Device and Package windows, validity and constraint type will be shown on the left side of the status bar. Site coordinates and type will

be shown in the right side.

- The task progress of a running task will be relocated to the right side of the status bar when the **Background** button is selected.

8. **Results Window Area** -The Results Window displays the status and results of commands in a set of windows grouped in the bottom of the Vivado IDE environment. As commands progress, messages are generated and log files and reports are created. The related information is shown here. The default windows are:

- **Messages** - Displays all messages for the active design.
- **Tcl Console** - Tcl commands can be entered here and a history of previous commands and outputs are also available.
- **Reports** - Quick access is provided to the reports generated throughout the design flow.
- **Log** -Displays the log files generated by the simulation, synthesis and implementation processes.
- **Design Runs** -Manages runs for the current project.

Additional windows that can appear in this area as required are: Find Results window, Timing Results window and Package Pins window.

With the layout of the Vivado IDE environment introduced, we can now move on to creating the Zynq system.

Exercise 1B Creating a Zynq System in Vivado

In this exercise we will be create a simple Zynq embedded system which implements a General Purpose Input/Output (GPIO) controller in the PL of the Zynq device on the ZedBoard. The GPIO controller will connect to the LEDs. It will also be connected to the Zynq processor via an AXI bus connection, allowing the LEDs to be controlled by a software application which we will create in Exercise 1C.

A graphical representation of the Zynq embedded design is provided in Figure 1.6.

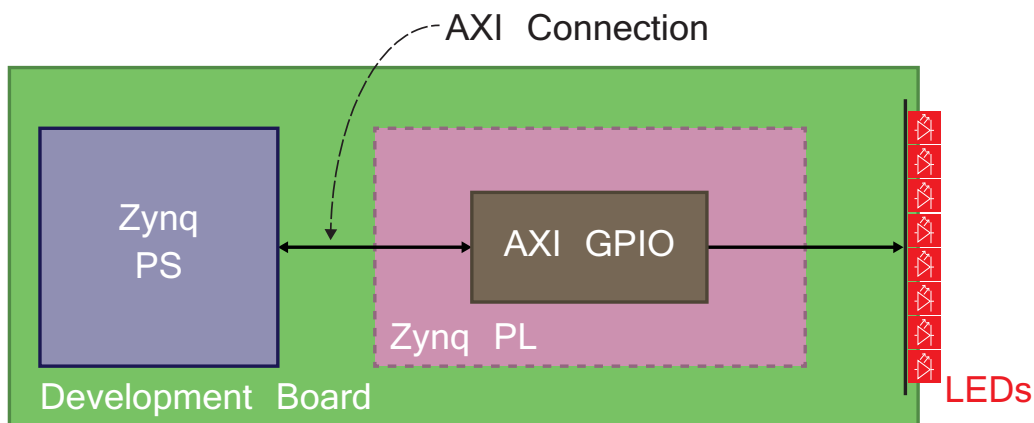


Figure 1.6: Zynq Embedded Design for Exercise 1B

We will begin by creating a new Block Design in Vivado IDE.

- (a) In the *Flow Navigator* window, select **Create Block Design** from the *IP Integrator* section, as in Figure 1.7:

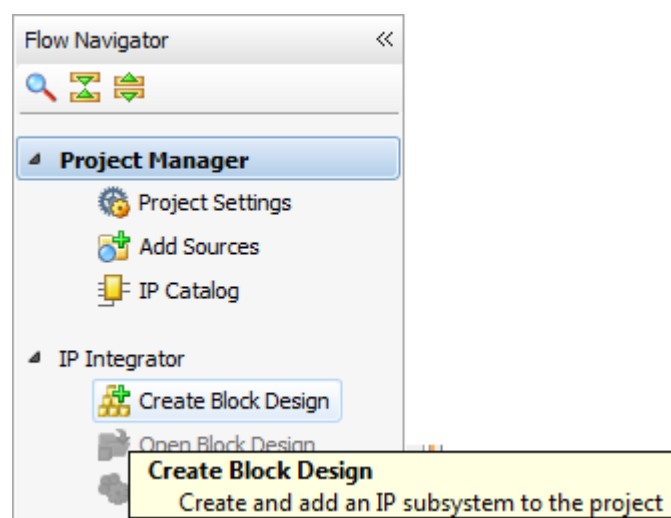


Figure 1.7: Creating a new Block Design in Flow Navigator

The *Create Block Design* dialogue will open.

- (b) Enter **first_zynq_system** in the Design name box, as in Figure 1.8:

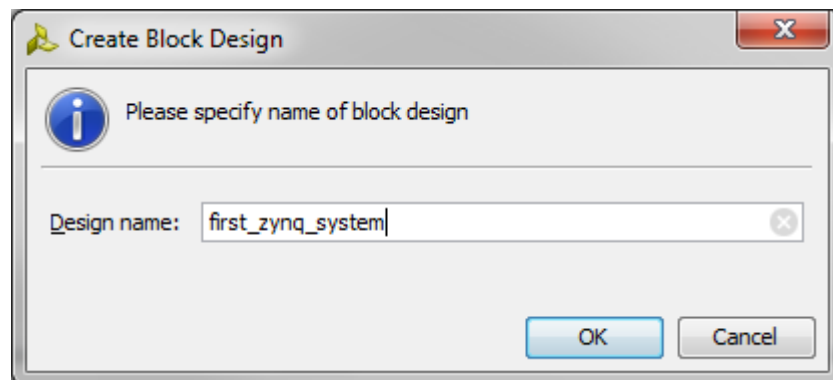


Figure 1.8: Create Block Design dialogue

Click **OK**. The *Vivado IP Integrator Diagram* canvas will open in the *Workspace*.

The first block that we will add to our design will be a Zynq Processing System.

- (c) In the *Vivado IP Integrator Diagram* canvas, right-click anywhere and select **Add IP**, as in Figure 1.9.

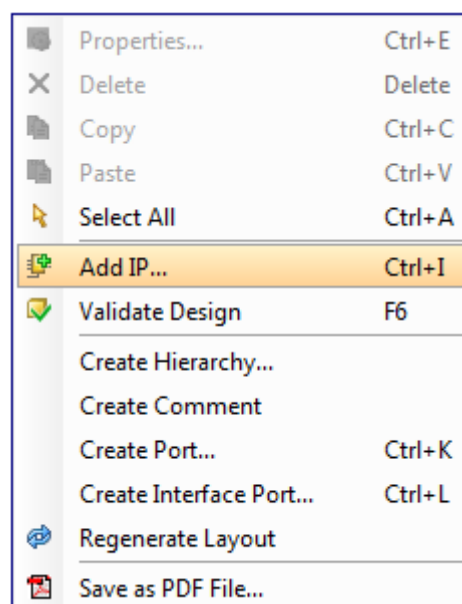


Figure 1.9: Add IP Option

Alternatively, select the **Add IP** option from the information message at the top of the canvas, shown in Figure 1.10.

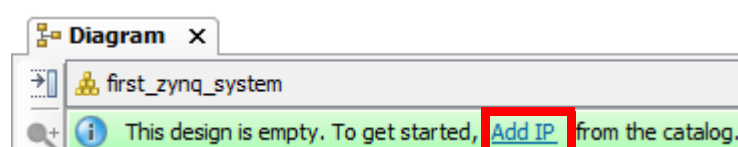


Figure 1.10: Add IP option in IP Integrator canvas information message

The pop-up IP Catalog window will open, as in Figure 1.11.

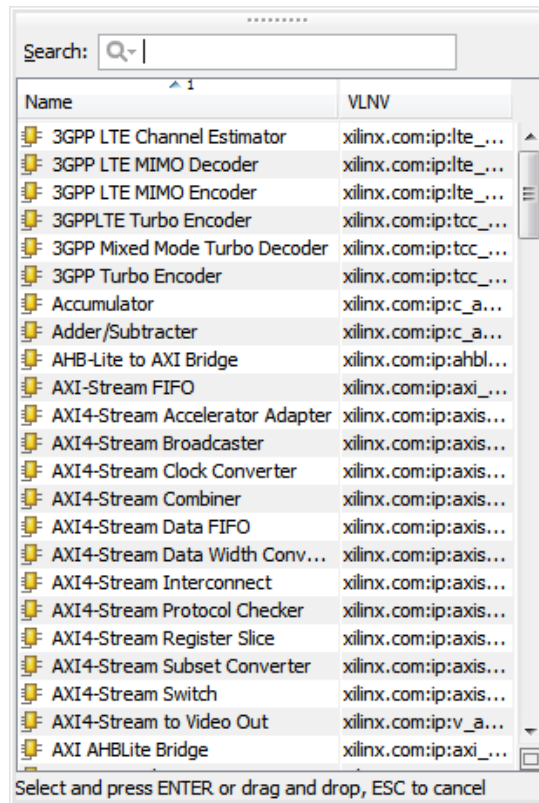


Figure 1.11: Pop-up IP Catalog Window

- (d) Enter **zynq** in the search field and select the **ZYNQ7 Processing System**, as shown in Figure 1.12, and press the **Enter** key on your keyboard.

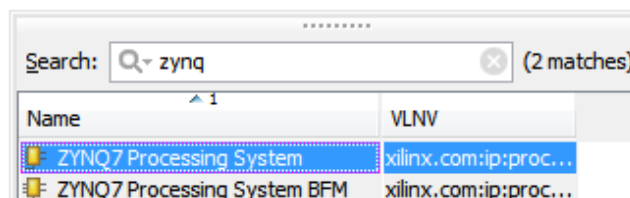


Figure 1.12: Adding ZYNQ7 Processing System from IP

You should see a similar message to the following in the *Tcl Console* window to confirm that the processing system has indeed been configured correctly:

```
create_bd_cell -type ip -vlnv xilinx.com:ip:processing_system7:5.4
processing_system7_0
```

Messages like this will be displayed in the *Tcl Console* window for all actions carried out on IP Integrator blocks.

The next step is to connect the **DDR** and **FIXED_IO** interface ports on the Zynq PS to the top-level interface ports on the design.

- (e) Click the **Run Block Automation** option from the *Designer Assistance* message at the top of the Diagram window and select **/processing_system7_0**, as shown in Figure 1.13.

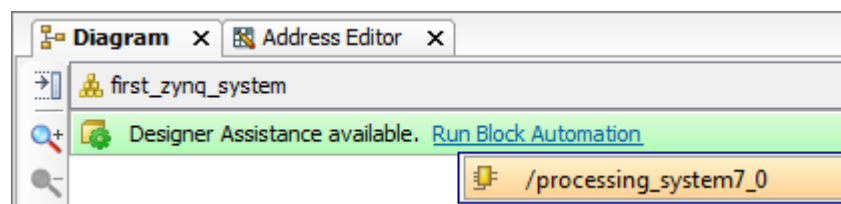


Figure 1.13: Run Block Automation - Processing System

You should notice that the selected item, in this case the ZYNQ7 Processing System, is highlighted in green.

Select **OK**, to generate the external connections for both the **DDR** and **FIXED_IO** interfaces, ensuring that the option to **Apply Board Preset** is selected.

Your block diagram should now resemble Figure 1.14.

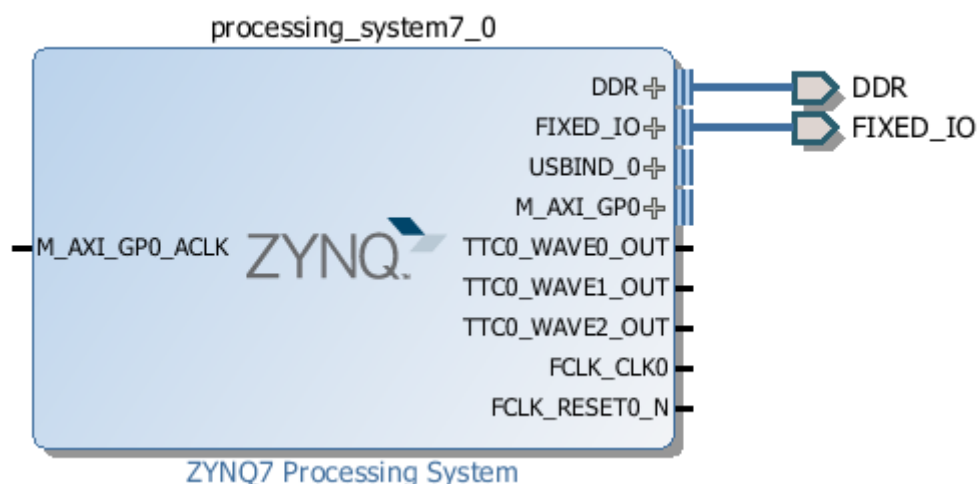


Figure 1.14: ZYNQ7 Processing System External Connections

As we are using the ZedBoard platform, and we specified this when creating the project, Vivado will configure the Zynq processor block accordingly.

Now that the main Zynq PS has been added to our design and configured, we can now add further blocks which will be placed in the PL to add functionality to the system. In this case we will only be adding a single block, **AXI GPIO**, to allow us to access the **LEDs** on the **ZedBoard**.

- (f) Right-click in an empty area of the *Diagram* window and select **Add IP**. Enter **GPIO** in the search field and add an instance of the **AXI GPIO** IP.

We will now use the *IP Integrator Designer Assistance* tool to automate the connection of the **AXI GPIO** block to the **ZYNQ7 Processing System**.

- (g) Click **Run Connection Automation** from the *Designer Assistance* message at the top of the *Diagram* window and select **/axi_gpio_0/S_AXI**, as shown Figure 1.15.

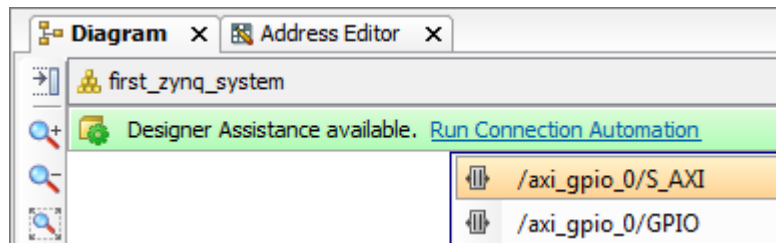


Figure 1.15: Run Block Automation - GPIO

This will automate the process of connecting the GPIO to an AXI port, and will automatically instantiate two further IP blocks:

- **Processor System Reset Module** - This provides customised resets for an entire processing system, including the peripherals, interconnect and the processor itself.
- **AXI Interconnect** - Provides an AXI interconnect for the system, allowing further IP and peripherals in the PL to communicate with the main processing system.

Leave the option for *Clock Connection (for unconnected clks)* to **Auto**, and Click **OK**.

All connections between the blocks should be made automatically.

One final connection is required to connect the **AXI GPIO** block to the **LEDs** on the ZedBoard. This can also be completed using *Designer Assistance*.

- (h) Click **Run Connection Automation** from the *Designer Automation* message at the top of the *Diagram* window and select **/axi_gpio_0/GPIO**.

The Run Connection Automation dialogue will open, as in Figure 1.16.

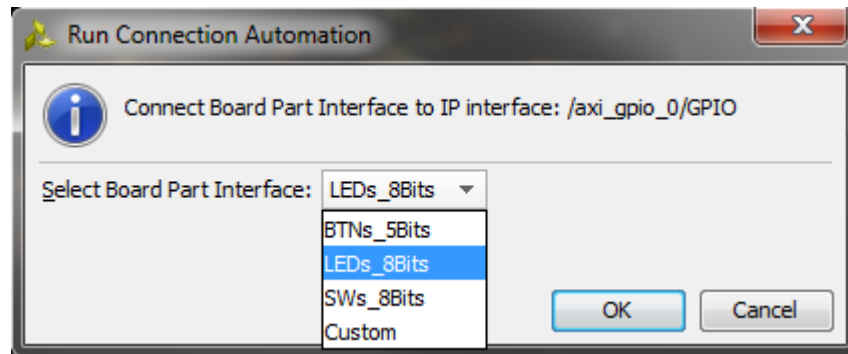


Figure 1.16: Run Connection Automation Dialogue - GPIO

Select **LEDs_8Bits** from the drop-down menu, and click **OK**.

The gpio interface of the AXI GPIO block will automatically be connected to the LEDs on the ZedBoard.

- (i) Click the *Regenerate Layout* button to tidy up the design schematic. Your complete design should resemble Figure 1.17.

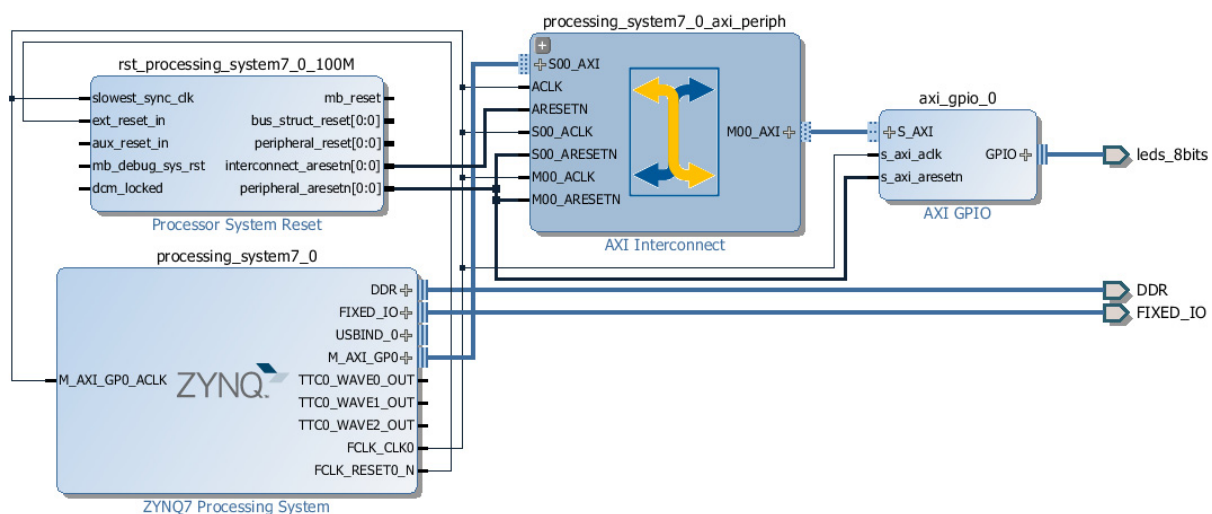


Figure 1.17: Zynq Processor System

The positions of the individual IP blocks in your design may vary slightly from Figure 1.17, but the blocks and their connections should be the same.

IP Integrator will automatically assign a memory map for all IP that is present in the design. We will not be changing the memory map in this tutorial, but for future reference we will take a look at the Address Editor.

- (j) Select the Address Editor tab from the top of the Workspace window, as shown in Figure 1.18, and expand the Data group.

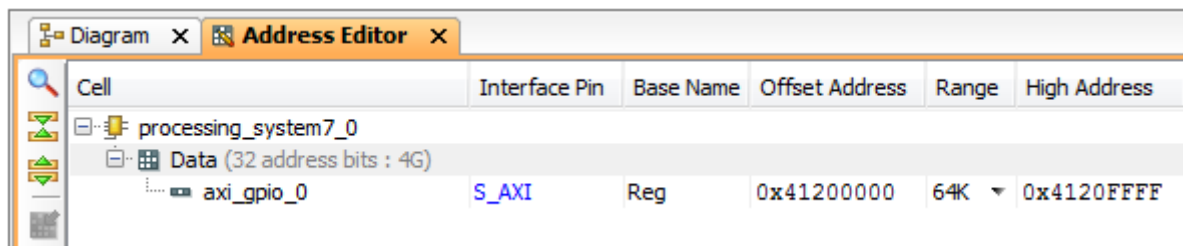


Figure 1.18: Address Editor Tab

You can see that IP Integrator has already assigned a memory map (the mapping of specific sections of memory to the memory-mapped registers of the IP blocks in the PL) to the AXI GPIO interface, and that it has a range of **64K**.

Now that our system is complete, we must first validate the design before generating the HDL design files.

- (k) Save your design by selecting **File > Save Block Design** from the *Menu Bar*.
- (l) Validate the design by selecting **Tools > Validate Design** from the *Menu Bar*. This will run a Design-Rule-Check (DRC).
Alternatively, select the Validate Design button, , from the Main Toolbar, or right-click anywhere in the *Diagram* canvas and select **Validate Design**.
- (m) A *Validate Design* dialogue should appear to confirm that validation of the design was successful. Click **OK**, to dismiss the message.

With the design successfully validated, we can now move on to generating the HDL design files for the system.

- (n) Switch to the **Sources Tab** by selecting **Window > Sources** from the *Menu Bar*.

- (o) Still in the **Sources** window, right-click on the top-level system design, which in this case is **first_zynq_system**, and select **Create HDL Wrapper**, as shown in Figure 1.19.

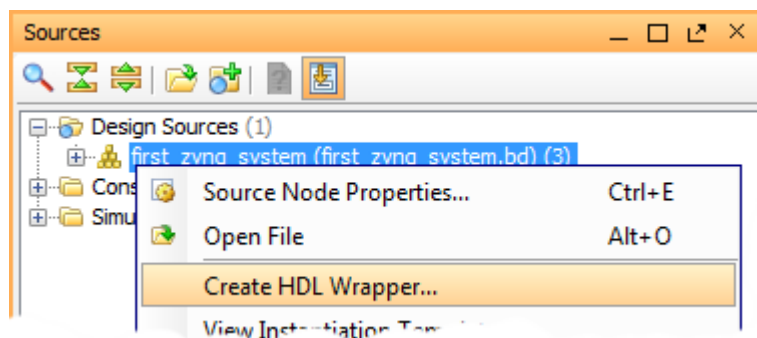


Figure 1.19: Create HDL Wrapper

The *Create HDL Wrapper* dialogue window will open. Select *Let Vivado manage wrapper and auto-update*, and click **OK**.

This will generate the top level HDL wrapper for our system.

All of the source files for the IP blocks that were used in the IP Integrator block diagram, as well as any relevant constraints files, will be generated during the synthesis process. As we specified VHDL as the target language when creating the project in Exercise 1A, all generated source files will be VHDL.

With all HDL design files generated, the next step in Vivado is to implement our design and generate a bitstream file.

- (p) In *Flow Navigator*, click **Generate Bitstream** from the *Program and Debug* section.

If a dialogue window appears prompting you to save your design, click **Save**.

- (q) A dialogue window will open requesting that you launch synthesis and implementation before starting the *Generate Bitstream* process. Click **Yes** to accept.

The combination of running the synthesis, implementation and bitstream generation processes back-to-back may take a few minutes, depending on the power of your computer system.

- (r) Once the bitstream generation is complete a dialogue window will open to inform you that the process has been completed successfully, as in Figure 1.20.

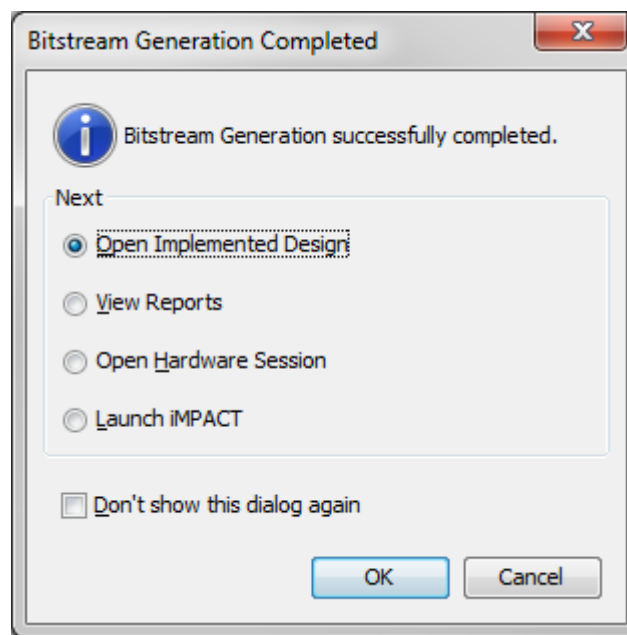


Figure 1.20: Bitstream Generation Completion Dialogue Window

Select **Open Implemented Design**, and click **OK**.

At this point you will be presented with the *Device* view, where you can see the PL resources which are utilised by the design.

With the bitstream generation complete, the building of the hardware image is complete. It must now be exported to a software environment where we will build a software application to control and interact with the custom hardware.

The final step in Vivado is to export the design to the SDK, where we will create the software application that will allow the Zynq PS to control the LEDs on the ZedBoard.

- (s) Select **File > Export > Export Hardware for SDK...** from the *Menu Bar*.

- (t) The *Export Hardware for SDK* dialogue window will open. Ensure that the options to **Include bitstream** and **Launch SDK** are selected, as in Figure 1.21, and click **OK**.

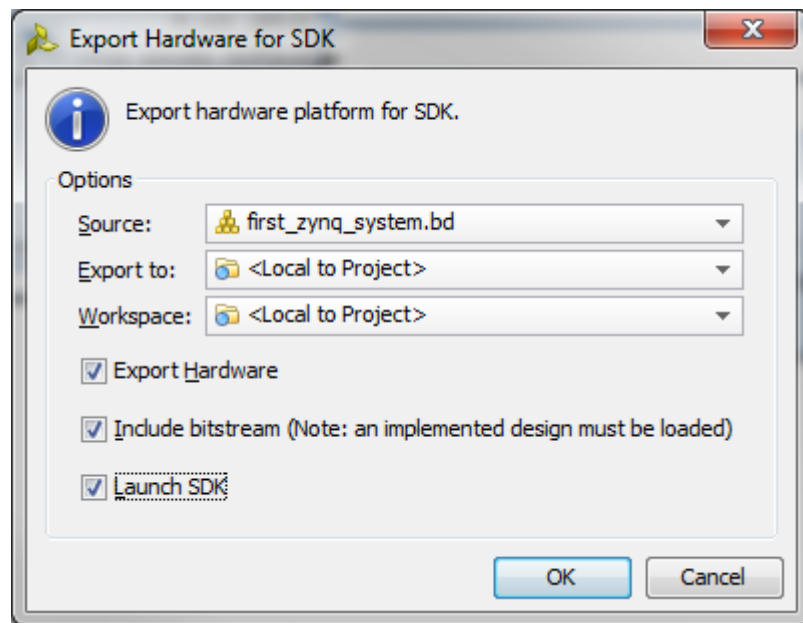


Figure 1.21: Export Hardware for SDK

NOTE: For the option to *Include bitstream* to be enabled, an implemented design must be active. This is the reason that we opened the implemented design in Step (r).

This concludes the steps that are required in Vivado IDE. All hardware components of the system have been configured and generated. In the next exercise we will move on to creating a simple software component which will control the system.

Exercise 1C Creating a Software Application in the SDK

In this exercise we will create a simple software application which will control the LEDs on the ZedBoard. The software application will run on the Zynq processing system and communicate with the AXI GPIO block which is implemented in the PL. We will take a look at the software drivers that are created by IP Integrator, for each of the IP modules, before building and executing the software on the ZedBoard.

The SDK should have opened after the conclusion of Exercise 1B. If it did not open, you can open the SDK by navigating to **Start > All Programs > Xilinx Design Tools > Vivado 2014.1 > Xilinx SDK 2014.1**

When launching the SDK from the start menu, you will need to specify the workspace that was created when the Vivado IP Integrator design was exported in Exercise 1B. It should be:

C:\Zynq_Book\first_zynq_design\first_zynq_design.sdk\SDK\SDK_Export

Enter this in the *Workspace* field of the *Workspace Launcher* dialogue window, as shown in Figure 1.22.

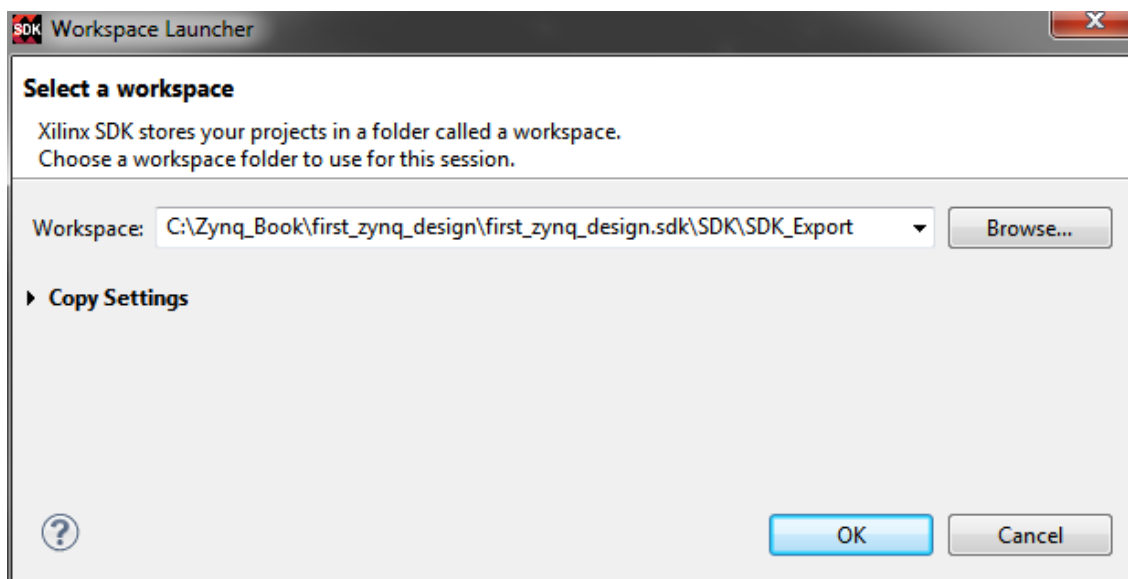


Figure 1.22: SDK Workspace Launcher Dialogue Window

With the SDK open, we can begin the creation of our software application.

- (a) Select **File > New > Application Project** from the *Menu bar*.

- (b) The *New Project* dialogue window will open. Enter **LED_test** in the *Project name* field, as shown in Figure 1.23, keeping all other options with the default settings. Click **Next**.

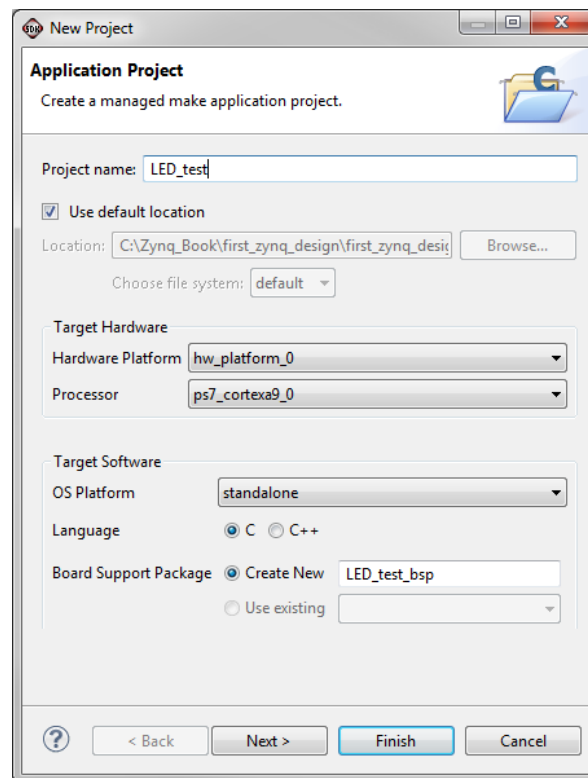


Figure 1.23: New Application Project Dialogue

- (c) At the *New Project Templates* screen, select **Empty Application**, as in Figure 1.24, and click **Finish** to create the project.

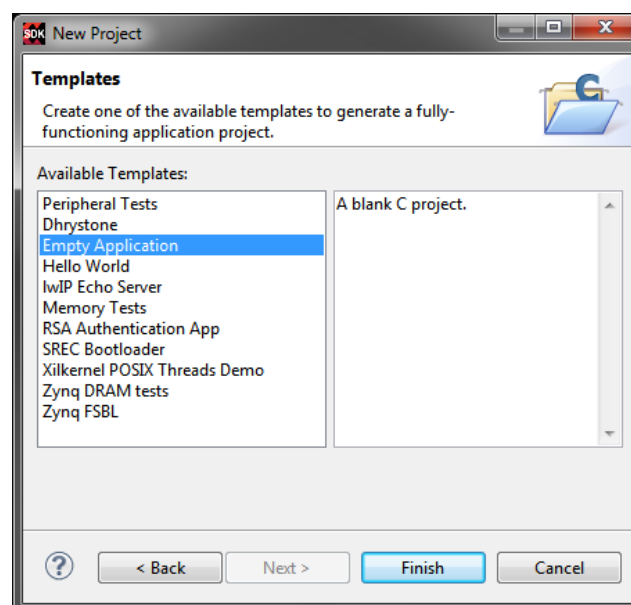


Figure 1.24: New Project Template Dialogue

NOTE: the new project should open automatically. If it doesn't, you may need to close the Welcome tab in order to view the project.

With the new Application Project created, we can now import some pre-prepared source code for the application.

- (d) In the *Project Explorer* panel, expand **LED_test** and highlight the *src* directory. Right-click and select **Import...**, as shown in Figure 1.25.

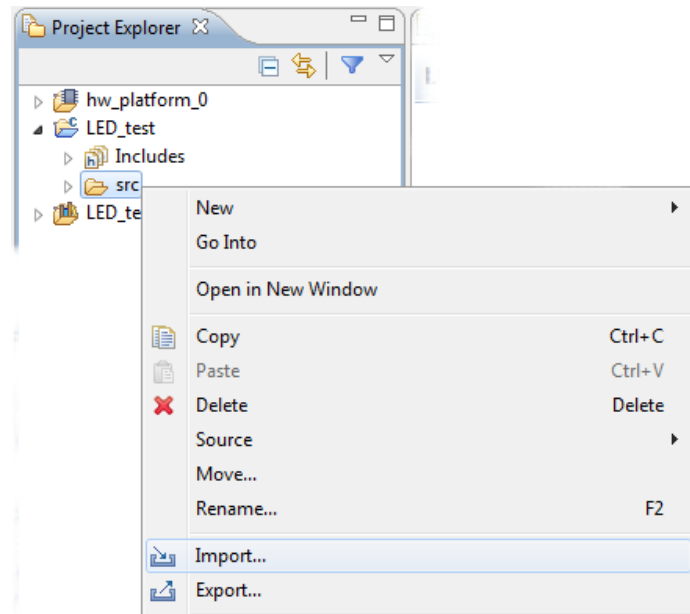


Figure 1.25: Import Source Files to Project

- (e) The *Import* window will open. Expand the **General** option and highlight **File System**, as in Figure 1.26, and click **Next**.

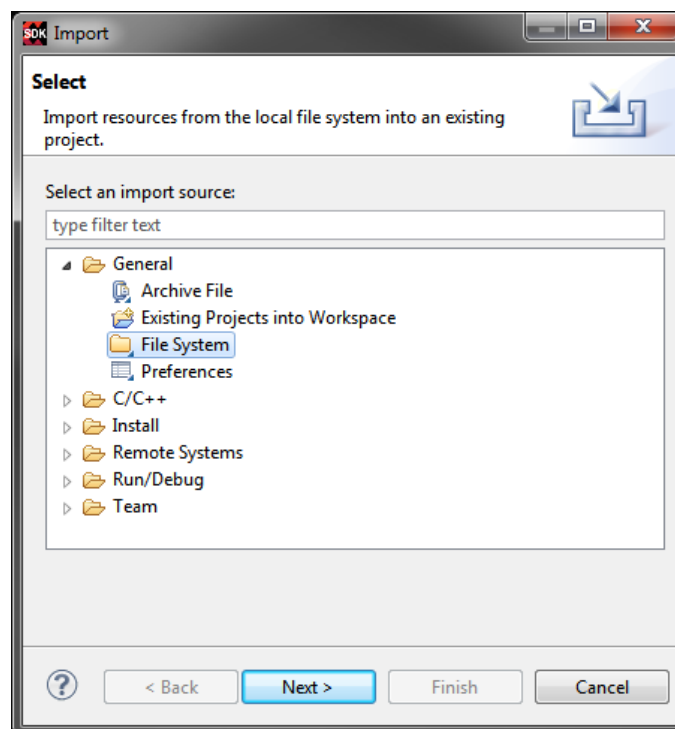


Figure 1.26: Import File System

- (f) In the *Import File System* window, click the **Browse...** button.
- (g) Navigate to the directory: **C:\Zynq_Book\sources\first_zynq_design** and click **OK**.
- (h) Select the file **LED_test_tut_1C.c**, as shown in Figure 1.27, and click **Finish**.

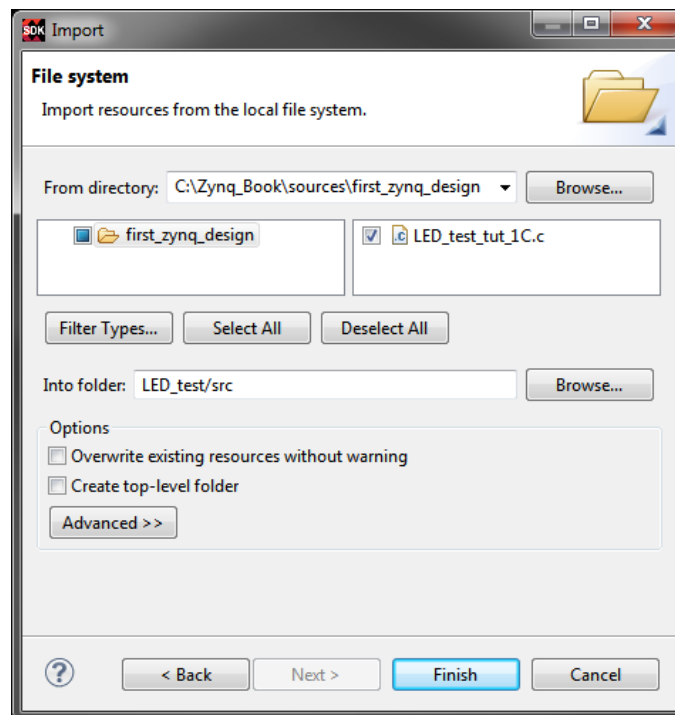


Figure 1.27: Import C Source File

The C source file will be imported and the project should automatically build. You should see a similar message to Figure 1.28 in the *Console* window.

```

CDT Build Console [LED_test]
12:04:12 **** Auto Build of configuration Debug for project LED_test ****
make all
'Building file: ../src/LED_test_tut_1C.c'
'Invoking: ARM gcc compiler'
arm-xilinx-eabi-gcc -Wall -O0 -g3 -c -fmessage-length=0 -I../LED_test_bsp/ps7_cortexa9_0/include
-MMD -MP -MF"src/LED_test_tut_1C.d" -MT"src/LED_test_tut_1C.d" -o "src/LED_test_tut_1C.o"
"../src/LED_test_tut_1C.c"
'Finished building: ../src/LED_test_tut_1C.c'
'Building target: LED_test.elf'
'Invoking: ARM gcc linker'
arm-xilinx-eabi-gcc -Wl,-T ../src/lscript.ld -L../LED_test_bsp/ps7_cortexa9_0/lib -o
"LED_test.elf" ./src/LED_test_tut_1C.o -Wl,--start-group,-lxil,-lgcc,-lc,--end-group
'Finished building target: LED_test.elf'
'Invoking: ARM Print Size'
arm-xilinx-eabi-size LED_test.elf |tee "LED_test.elf.size"
  text    data    bss     dec     hex filename
23964    1120    29812   54896   d670 LED_test.elf
'Finished building: LED_test.elf.size'
12:04:13 Build Finished (took 891ms)

```

Figure 1.28: Build Finished Console Message

- (i) Open the imported source file by expanding the `src` folder and double-clicking on **LED_test_tut_1C.c**, and explore the code.

Note the command `XGpio_Initialize(&Gpio, GPIO_DEVICE_ID);` This is a function provided by the GPIO device driver in the file `xgpio.h`. It initialises the XGpio instance, *Gpio*, with the unique ID of the device specified by `GPIO_DEVICE_ID`.

If you look toward the top of the source file you will see that `GPIO_DEVICE_ID` is defined as `XPAR_AXI_GPIO_0_DEVICE_ID`. The value of `XPAR_AXI_GPIO_0_DEVICE_ID` can be found by opening the file, `xparameters.h`, which is automatically generated by Vivado IDE when exporting a hardware design to the SDK. It contains definitions of all the hardware parameters of the system.

The function, `XGpio_SetDataDirection(&Gpio, LED_CHANNEL, 0xFF);` is also provided by the GPIO device driver, and sets the direction of the specified GPIO port. As we are specifying the LEDs in this case, it is specifying an output. Bits set to '0' are output, and bits set to '1' are input. As there are 8 LEDs, by setting the LED channel direction to a value of `0x00`, or `00000000` in binary, we are setting all 8 LEDs as outputs.

Further information on the peripheral drivers can be found by selecting the **system.mss** tab. A list of all the peripherals in the system is provided, along with links to available documentation and examples, as shown in Figure 1.29.

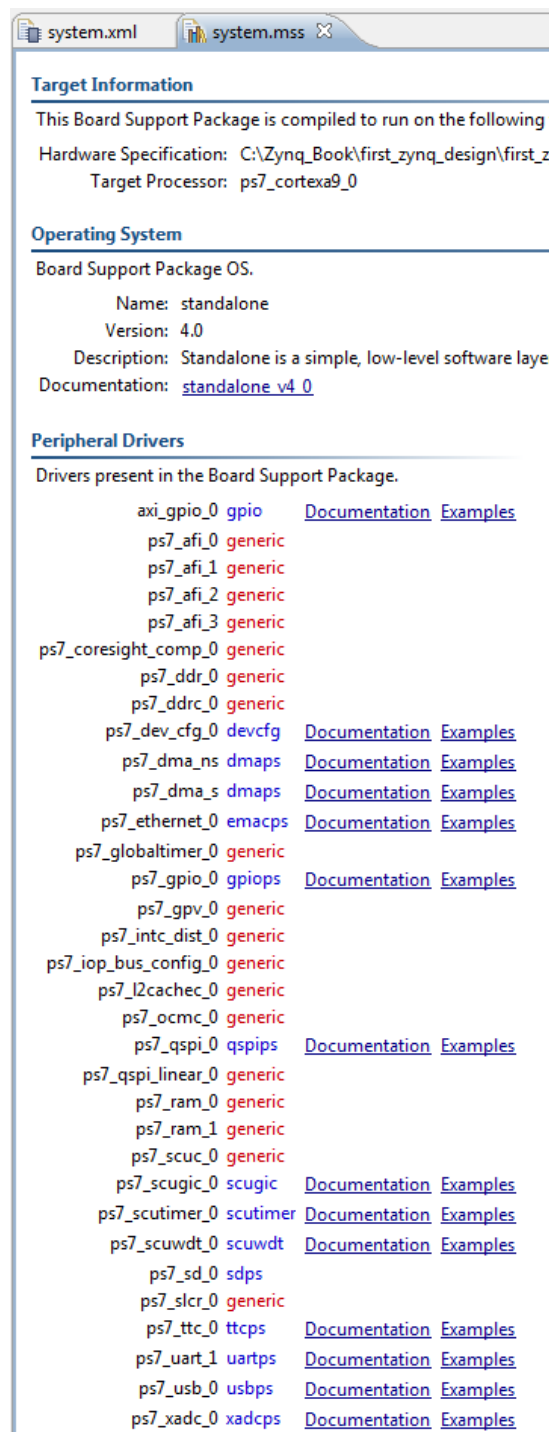


Figure 1.29: Peripheral Documentation and Drivers in system.mss tab

The next step is to program the Zynq PL with the bitstream file that we generated in Exercise 1B.

Ensure that the ZedBoard is powered on and that the JTAG port is connected to the PC via the provided USB-A to USB-B cable.

- (j) Download the bitstream to the Zynq PL by selecting **Xilinx Tools > Program FPGA** from the *Menu bar*. The *Program FPGA* window will appear. The Bitstream field should already be populated with the correct bitstream file, as in Figure 1.30.

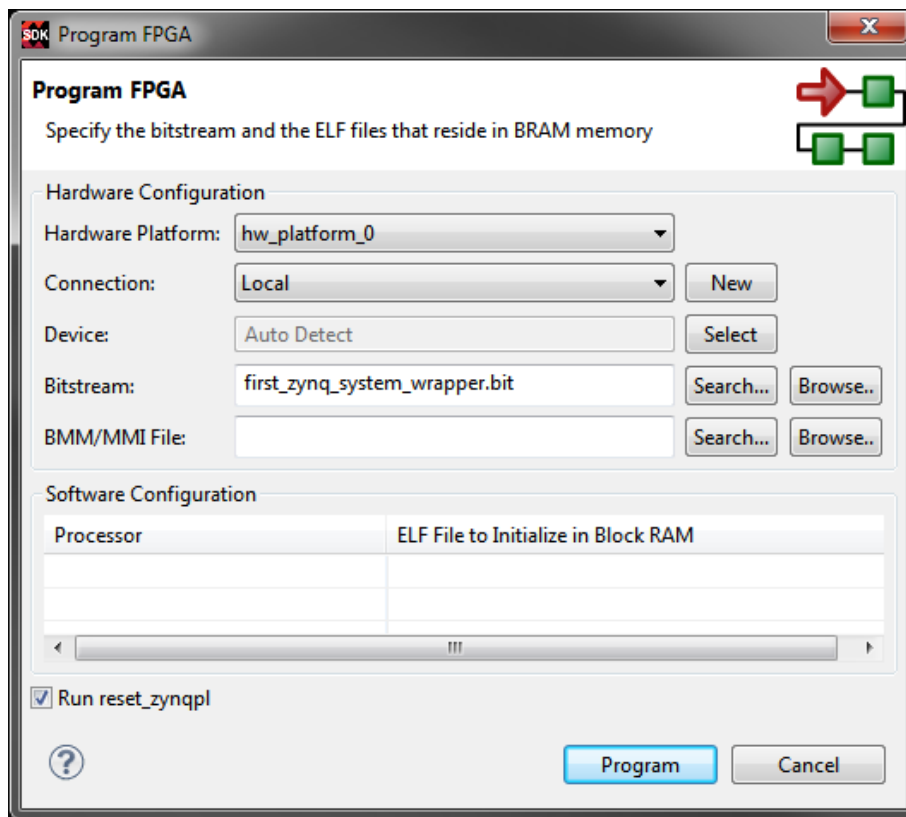


Figure 1.30: Program FPGA dialogue Window

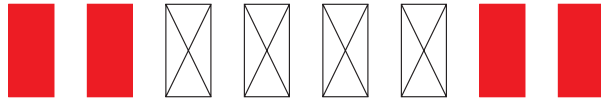
NOTE: Once the device has successfully been programmed, the *DONE LED* on the ZedBoard will turn blue.

With the Zynq PL successfully configured with the bitstream file, we can now launch our software application on the Zynq PS.

- (k) Select the project **LED_test** in *Project Explorer*. Right-click and select **Run As > Launch on Hardware (GDB)**.

After a few seconds the LEDs on the ZedBoard should begin to flash between the states highlighted in Figure 1.31.

State A:



State B:



Figure 1.31: LED Flashing States

You have successfully created and executed your first software application on the Zynq processing system.
